

Banco de Dados I

Ronierison Maciel

Senac

Fevereiro de 2026



Nome: Roni

Formação: Mestre em Ciência da Computação

Ocupação: Pesquisador, Professor e Escovador de bits

Hobbies: Jogar cartas, ficar com a família

Interesses: Carros, DevSec, DS e ML

Email: ronierison.maciел@pe.senac.br

GitHub: <https://github.com/0x03c1>

- 1 Módulo 1 – Introdução a Bancos de Dados e MySQL
- 2 Módulo 2 – Modelo Relacional e SQL Básico (DDL/DML)
- 3 Módulo 3 – DQL: Filtros, Operadores Lógicos e Ordenação
- 4 Módulo 4 – DQL Avançado: Agregação, GROUP BY, HAVING e Subconsultas
- 5 Módulo 5 – Modelagem Conceitual e Lógica (MER/DER → Tabelas)
- 6 Módulo 6 – Junção de Tabelas (JOINS) em MySQL
- 7 Módulo 7 – Normalização de Dados (1FN, 2FN, 3FN)
- 8 Módulo 8 – Estrutura de Armazenamento (visão geral)
- 9 Módulo 9 – Indexação (introdução)
- 10 Módulo 10 – Encerramento e Ponte para BD II

O escopo de Banco de Dados I

Em **BD I** concentramos esforços em: modelagem (conceitual/lógica), projeto de tabelas, integridade, **DDL**, **DML** e — principalmente — **DQL** (consultas), incluindo **JOINS**, **agregações** e **subconsultas básicas**.

O que **NÃO** veremos aqui (fica para BD II)

- **Programação em banco:** *stored procedures, functions, triggers, cursors*.
- **Views** e mecanismos de segurança avançados.
- **Controle de transações avançado** (níveis de isolamento, bloqueios, deadlocks) e administração do SGBD.

A ideia é construirmos uma base *sólida* de SQL declarativo antes de avançarmos para tópicos procedurais e de administração.

Tópicos:

- Visão geral sobre BD e SGBD, instalação do MySQL
- Modelos de Dados, esquemas e arquiteturas
- Linguagens e interfaces de SGBDs, criação de tabelas

Objetivo:

- Introduzir os conceitos fundamentais de Banco de Dados e realizar a configuração inicial do MySQL

O que são Bancos de Dados (BD)?

Um Banco de Dados é uma coleção organizada de dados, tipicamente armazenados e acessíveis eletronicamente.

- **Exemplo:** Catálogo de produtos de uma loja, lista de alunos de uma escola.

CLIENTE	
CODIGO	NOME
1234	CARLOS
5678	JOÃO
9101	PEDRO
1213	MARIA

VENDEDOR	
CODIGO	NOME
11	CARMEM
12	DJANIRA
13	ZECA
14	MARIO

PRODUTO	
CODIGO	DESCRICAO
123	LAPIS
456	CANETA
789	PAPELA4
101	TESOURA
123	BORRACHA
141	LIVRO

CONTÉM		
PEDIDO	PRODUTO	QUANTIDADE
100/05	123	10
100/05	789	20
101/05	456	30
102/05	456	40
103/05	101	50
103/05	121	60
103/05	141	70
104/05	456	80

PEDIDO			
NUMERO	DATA	VENDEDOR	CLIENTE
100/05	01/01/05	12	5678
101/05	01/02/05	11	9101
102/05	01/03/05	13	1213
103/05	01/04/05	14	1234
104/05	01/05/05	12	1213

Figura: Tabelas

O que é um Sistema de Gerenciamento de Banco de Dados (SGBD)?

Conteúdo:

- **Definição:** Um SGBD é um software que permite a criação, gestão, manipulação e controle de acesso a bancos de dados.
- **Funções:** Controle de concorrência, recuperação de falhas, segurança e integridade de dados.

Exemplos: MySQL, PostgreSQL, Oracle, SQL Server.

Por que Bancos de Dados são importantes?

- Centralização e organização dos dados.
- Suporte à tomada de decisão.
- Eficiência e escalabilidade em operações de negócio.

Casos de Uso: Comércio eletrônico, sistemas de gerenciamento de clientes (CRM), sistemas de controle financeiro.

Visão geral dos SGBDs mais usados

- **MySQL:** Open source, amplamente usado em aplicações web.
- **PostgreSQL:** Avançado, com suporte a tipos de dados complexos.
- **Oracle:** Focado em grandes empresas, oferece alta performance e segurança.
- **SQL Server:** Solução da Microsoft, integrada com outras ferramentas da empresa.

Gráfico Comparativo: Popularidade dos SGBDs (baseado em pesquisas recentes).

Você Sabia?

O mercado global de bancos de dados movimentou mais de **US\$ 100 bilhões em 2024**, com projeção de crescimento anual de 14% até 2030. O MySQL é utilizado por mais de **39% dos desenvolvedores** no mundo, segundo a Pesquisa Stack Overflow Developer Survey 2024.

Dados do Mundo Real

- O **Facebook** gerencia mais de **600 terabytes** de dados novos por dia em seus bancos MySQL Meta Engineering Blog – MySQL at Meta.
- O **Banco do Brasil** processa mais de **70 milhões de transações diárias** usando SGBDs relacionais Relatório Anual Banco do Brasil.
- Empresas que adotam SGBDs modernos reduzem em até **40%** o tempo de resposta em consultas Gartner – Database Management Systems Report.

Como instalar o MySQL

Passos:

- Baixar o MySQL Community Server do site oficial.
- Executar o instalador e seguir as instruções.
- Configurar a senha do root e as opções de segurança.
- Verificar a instalação via terminal.

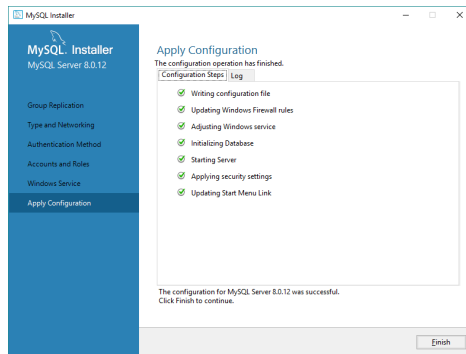


Figura: Instalação do MySQL

O que é MySQL Workbench?

Ferramenta GUI para modelagem de dados, desenvolvimento SQL e administração de servidores.

- Conectar ao servidor MySQL.
- Criar um banco de dados.
- Explorar as funcionalidades básicas.

Introdução aos Modelos de Dados

- **Modelos hierárquicos:** Dados organizados em uma estrutura de árvore.
- **Modelos em rede:** Dados organizados em gráficos, permitindo múltiplas relações.
- **Modelos relacionais:** Dados organizados em tabelas, a base do MySQL.

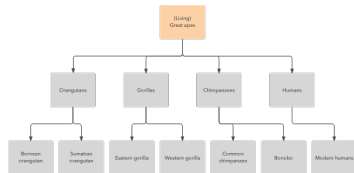


Figura: Hierárquico

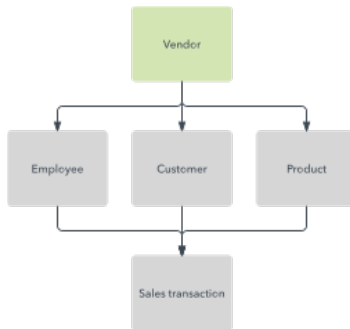


Figura: Rede

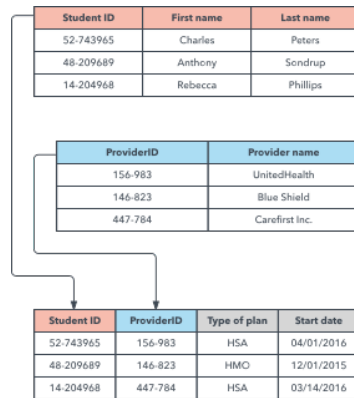


Figura: Relacional

Linha do Tempo

- **1960s**: Modelo Hierárquico – IBM IMS, usado pela NASA no programa Apollo IBM IMS – History.
- **1970**: Edgar F. Codd publica o artigo seminal sobre o **Modelo Relacional** Codd (1970) – ACM Digital Library.
- **1980s**: SGBDs relacionais dominam o mercado (Oracle, DB2) Oracle Database – A Brief History.
- **2000s**: Surgem os bancos **NoSQL** (MongoDB, Cassandra) para Big Data MongoDB – History of NoSQL.
- **2020s**: Bancos **NewSQL** e multimodelo ganham força (CockroachDB, Fauna) DB-Engines Ranking – Database Popularity.

Curiosidade

O artigo de Codd, “*A Relational Model of Data for Large Shared Data Banks*” (1970), é considerado um dos documentos mais influentes da história da computação e deu origem ao SQL Codd (1970) – ACM Digital Library – leia o artigo original.

Estrutura dos Bancos de Dados

- **Esquema:** A estrutura lógica de um banco de dados, definindo como os dados são organizados e inter-relacionados.

Arquiteturas:

- **Monolítica:** Todos os dados e serviços estão centralizados em um único sistema.
- **Cliente-Servidor:** Dados armazenados em servidores, acessados por clientes.
- **Distribuída:** Dados espalhados por múltiplos sistemas interconectados.

Linguagens e interfaces de SGBDs

Sublinguagens do SQL:

- **DDL** (*Data Definition Language*): cria e altera estruturas — CREATE, ALTER, DROP.
- **DML** (*Data Manipulation Language*): manipula dados — INSERT, UPDATE, DELETE.
- **DQL** (*Data Query Language*): consulta dados — SELECT (foco principal de BD I).
- **DCL** (*Data Control Language*): controla permissões — GRANT, REVOKE (*visto rapidamente*).
- **TCL** (*Transaction Control Language*): básico de transação — COMMIT, ROLLBACK.

Interfaces:

- **CLI** (Command-Line Interface): Interação via comandos de texto.
- **GUI** (Graphical User Interface): Interação via interface gráfica, como o MySQL Workbench.

```
1 -- Exemplo de DDL
2 ALTER TABLE clientes ADD telefone VARCHAR(15);
3
4 -- Exemplo de DML
5 SELECT nome, email FROM clientes WHERE data_cadastro > '2026-01-01';
```


Definindo tabelas e tipos de dados

Tipos de Dados:

- Numéricos (INT, FLOAT, DECIMAL).
- Texto (CHAR, VARCHAR, TEXT).
- Data/Hora (DATE, DATETIME, TIMESTAMP).
- Booleano/Lógico (BOOLEAN – alias de TINYINT(1)).

Exemplo de criação de tabela:

```
1 CREATE TABLE alunos (  
2     id INT AUTO_INCREMENT PRIMARY KEY,  
3     nome VARCHAR(100) NOT NULL,  
4     data_nascimento DATE,  
5     email VARCHAR(120) UNIQUE  
6 );
```

As constraints são regras automáticas para manter os dados consistentes:

- PRIMARY KEY: identificador único da linha.
- FOREIGN KEY: referência a uma chave primária de outra tabela.
- NOT NULL: campo obrigatório.
- UNIQUE: valor não pode se repetir.
- DEFAULT: valor padrão se não informado.
- CHECK: validação customizada (ex.: idade ≥ 0).

```
1 CREATE TABLE produtos (  
2     id             INT AUTO_INCREMENT PRIMARY KEY,  
3     nome           VARCHAR(100) NOT NULL,  
4     preco          DECIMAL(10,2) NOT NULL DEFAULT 0.00,  
5     estoque        INT DEFAULT 0,  
6     CONSTRAINT chk_preco CHECK (preco  $\geq$  0),  
7     CONSTRAINT chk_estoque CHECK (estoque  $\geq$  0)  
8 );
```

Atividade da Aula

Tempo estimado: 30 minutos – realizar no MySQL Workbench.

- 1 Crie um banco loja_aula (`CREATE DATABASE loja_aula;`).
- 2 Selecione o banco com `USE loja_aula;`.
- 3 Crie a tabela categorias com id (PK auto-incremento) e nome (VARCHAR 50, NOT NULL, UNIQUE).
- 4 Crie a tabela produtos com id, nome, preco DECIMAL(10,2) CHECK(preco>=0) e categoria_id (FK).
- 5 Insira 3 categorias e 6 produtos.
- 6 Liste tudo com `SELECT * FROM produtos;` e `SELECT * FROM categorias;`.

Entrega

Print da tela do Workbench mostrando o resultado dos dois SELECT, mais o arquivo .sql com todos os comandos.

O que vamos fazer na próxima semana?

- Revisão dos conceitos básicos de SQL.
- Introdução ao modelo de dados relacional.
- Primeiras operações com SQL (SELECT, INSERT, UPDATE, DELETE).

Tópicos:

- Conceitos do modelo de dados relacional e tabelas relacionais.
- Operações básicas em SQL (INSERT, SELECT, UPDATE, DELETE).
- Introdução ao bloco transacional simples (COMMIT/ROLLBACK).

Objetivo:

- Entender o modelo relacional e realizar operações básicas em SQL no MySQL.

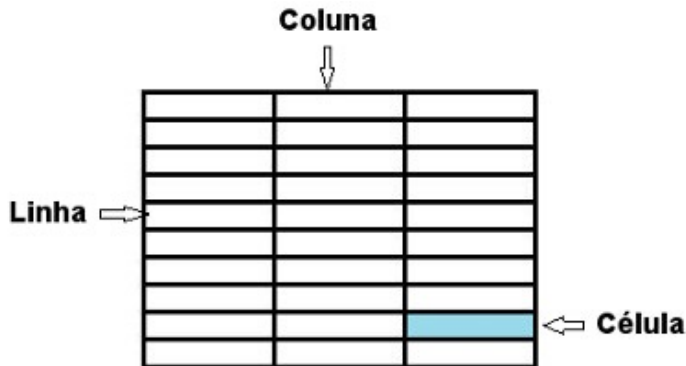
O que é o modelo de dados relacional?

Conceito:

- Modelo de dados que organiza informações em tabelas (também conhecidas como relações).

Elementos-chave:

- **Tabelas:** Conjuntos de dados organizados em linhas e colunas.
- **Linhas** (Tuplas): Cada linha representa um registro único.
- **Colunas** (Atributos): Cada coluna representa um campo de dado dentro de um registro.



Estrutura de uma tabela relacional

- **Chave primária:** Coluna ou conjunto de colunas que identifica de forma única cada registro em uma tabela.
- **Chave estrangeira:** Coluna que cria uma ligação entre duas tabelas diferentes, referenciando a chave primária de outra tabela.
- **Índices:** Estruturas que melhoram a velocidade de operações de consulta nas tabelas.

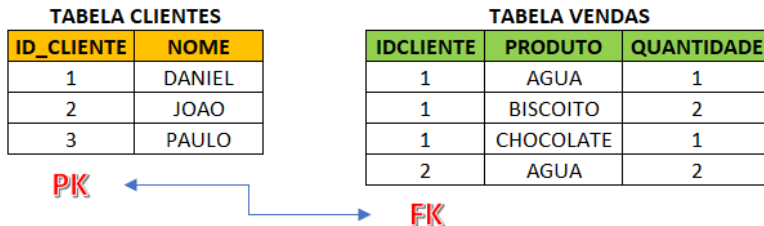


Figura: Foreign Key (FK) and Primary Key (PK)

Criando tabelas e definindo relações

Exemplo de SQL:

```
1 CREATE TABLE departamentos (  
2     id INT AUTO_INCREMENT PRIMARY KEY,  
3     nome VARCHAR(100) NOT NULL  
4 );  
5  
6 CREATE TABLE empregados (  
7     id INT AUTO_INCREMENT PRIMARY KEY,  
8     nome VARCHAR(100) NOT NULL,  
9     salario DECIMAL(10,2),  
10    data_admissao DATE,  
11    departamento_id INT,  
12    FOREIGN KEY (departamento_id) REFERENCES departamentos(id)  
13 );
```

Prática: Criar tabelas e estabelecer relações no MySQL Workbench.

Introdução às operações básicas em SQL

- INSERT: Inserção de novos registros.
- SELECT: Consulta de dados.
- UPDATE: Atualização de registros existentes.
- DELETE: Exclusão de registros.

Inserindo dados em tabelas

```
1 INSERT INTO <tabela> (<coluna1>, <coluna2>, <coluna3>)  
2 VALUES (<valor1>, <valor2>, <valor3>);
```

Inserção de múltiplos registros

```
1 INSERT INTO <tabela> (<coluna1>, <coluna2>)  
2 VALUES  
3     (<valor1>, <valor2>),  
4     (<valor3>, <valor4>),  
5     (<valor5>, <valor6>);
```

Exemplo concreto – carregando departamentos:

```
1 INSERT INTO departamentos (nome) VALUES  
2     ('Tecnologia'),  
3     ('Recursos Humanos'),  
4     ('Financeiro'),  
5     ('Marketing');
```

Estrutura básica de uma consulta SQL

```
1 SELECT <coluna1>, <coluna2>  
2 FROM <tabela>  
3 WHERE <condicao>;
```

Principais cláusulas:

- WHERE – filtra linhas de acordo com uma condição.
- GROUP BY – agrupa registros com valores iguais.
- HAVING – filtra resultados após o agrupamento.
- ORDER BY – ordena os resultados da consulta.
- LIMIT – restringe o número de linhas retornadas.

Atualizando registros em uma tabela

```
1 UPDATE <tabela>  
2 SET <coluna1> = <valor1>, <coluna2> = <valor2>  
3 WHERE <condicao>;
```

Observações:

- SET define os novos valores das colunas.
- WHERE determina quais registros serão atualizados.
- Sem WHERE, **todas as linhas da tabela serão modificadas.**

Removendo registros de uma tabela

```
1 DELETE FROM <tabela>  
2 WHERE <condicao>;
```

Observações:

- DELETE remove linhas da tabela.
- WHERE define quais registros serão excluídos.
- Sem WHERE, **todas as linhas da tabela serão removidas.**

Transação na visão de BD I

Em BD I usamos um bloco transacional como uma *caixa de segurança*: só confirmamos o que acontece dentro do bloco se tudo correr bem. Estudo aprofundado de **níveis de isolamento**, **bloqueios** e **deadlocks** fica para **Banco de Dados II**.

Comandos básicos (apenas estes três):

- **START TRANSACTION**: Inicia uma nova transação.
- **COMMIT**: Confirma as mudanças realizadas pela transação.
- **ROLLBACK**: Reverte as mudanças realizadas pela transação.

Exemplo:

```
1 START TRANSACTION;  
2 UPDATE empregados SET salario = salario * 1.1 WHERE departamento_id = 1;  
3 -- conferir o resultado com SELECT antes de fechar:  
4 SELECT id, nome, salario FROM empregados WHERE departamento_id = 1;  
5 COMMIT;      -- confirma; OU use ROLLBACK; para descartar
```

Case: Nubank e o Poder do SQL

O Nubank processa mais de **1,5 bilhão de eventos por dia**. Toda operação de crédito, débito e transferência passa por transações SQL que garantem a consistência financeira Nubank Building – Data Engineering Blog. Um simples erro em um UPDATE sem WHERE custou a uma fintech brasileira mais de **R\$ 2 milhões** em 2022 TecMundo – Banco de Dados: boas práticas.

Dica Profissional – regra de ouro do BD I

- **Nunca** execute UPDATE ou DELETE sem WHERE em produção.
- Sempre use START TRANSACTION + ROLLBACK ao testar alterações.
- SELECT antes do UPDATE/DELETE é uma prática de segurança essencial.

Exercícios (resolva no seu MySQL local)

Considere as tabelas departamentos(id, nome) e empregados(id, nome, salario, data_admissao, departamento_id).

- 1 Crie e popule as tabelas com pelo menos 4 departamentos e 10 empregados.
- 2 Escreva um INSERT que insira *na mesma instrução* 3 empregados de uma vez.
- 3 Aumente em 5% o salário de todos os empregados do departamento “Tecnologia”.
- 4 Remova os empregados admitidos antes de 2020-01-01.
- 5 Refaça o item 4 dentro de START TRANSACTION ... ROLLBACK, e mostre que os dados continuam intactos.

Atividade da Aula – 40 minutos

A partir do laboratório 1, evolua o esquema:

- ➊ Adicione a tabela `clientes(id, nome, email UNIQUE, cidade)`.
- ➋ Adicione a tabela `pedidos(id, cliente_id FK, data_pedido, total)`.
- ➌ Insira 5 clientes e 8 pedidos, distribuindo entre os clientes.
- ➍ Atualize o `total` de um pedido específico via `UPDATE`.
- ➎ Use `START TRANSACTION` para excluir um pedido e em seguida `ROLLBACK` para verificar o comportamento.

Entrega: arquivo `.sql` com todos os comandos comentados.

Tópicos:

- Aprofundar a cláusula WHERE com **operadores lógicos** e relacionais.
- Filtros avançados: AND, OR, NOT, IN, BETWEEN, LIKE, IS NULL.
- Ordenação com ORDER BY e limitação com LIMIT/OFFSET.
- Uso de DISTINCT e aliases (AS).

Comparam dois valores e retornam verdadeiro/falso.

- = igual a, <> ou != diferente de.
- >, <, >=, <=.

```
1 SELECT * FROM empregados WHERE salario >= 3000;  
2 SELECT * FROM empregados WHERE departamento_id <> 2;  
3 SELECT * FROM empregados WHERE data_admissao > '2024-01-01';
```

Combinam múltiplas condições no WHERE.

```
1 -- AND: TODAS as condições verdadeiras
2 SELECT * FROM empregados
3 WHERE salario > 3000 AND departamento_id = 1;
4
5 -- OR: PELO MENOS UMA condição verdadeira
6 SELECT * FROM empregados
7 WHERE departamento_id = 1 OR departamento_id = 3;
8
9 -- NOT: nega a condição
10 SELECT * FROM empregados
11 WHERE NOT departamento_id = 2;
```

Atenção: precedência

AND tem precedência maior que OR. Em dúvida, use parênteses:

```
1 SELECT * FROM empregados
2 WHERE (departamento_id = 1 OR departamento_id = 3)
3     AND salario > 3000;
```

Atalhos legíveis para condições compostas.

```
1 -- IN: pertence a uma lista
2 SELECT * FROM empregados
3 WHERE departamento_id IN (1, 2, 5);
4
5 -- BETWEEN: dentro de um intervalo (inclusivo)
6 SELECT * FROM empregados
7 WHERE salario BETWEEN 2000 AND 5000;
8
9 -- NOT IN
10 SELECT * FROM empregados
11 WHERE departamento_id NOT IN (4);
```

Equivalências didáticas

- $x \text{ IN } (1, 2, 3) \equiv x=1 \text{ OR } x=2 \text{ OR } x=3.$
- $x \text{ BETWEEN } a \text{ AND } b \equiv x \geq a \text{ AND } x \leq b.$

Usado para padrões parciais em texto. Os *coringas* são:

- % (percentual): zero ou mais caracteres quaisquer.
- _ (underscore): exatamente um caractere qualquer.

```
1 -- Começa com 'A'
2 SELECT * FROM empregados WHERE nome LIKE 'A%';
3
4 -- Termina com 'Silva'
5 SELECT * FROM empregados WHERE nome LIKE '%Silva';
6
7 -- Contém 'an'
8 SELECT * FROM empregados WHERE nome LIKE '%an%';
9
10 -- 4 letras, segunda letra 'a'
11 SELECT * FROM empregados WHERE nome LIKE '_a__';
```

NULL não é zero, nem string vazia — é ausência de valor. Comparações com = *nunca* casam com NULL.

```
1 -- Empregados que ainda nao tem departamento atribuido
2 SELECT * FROM empregados WHERE departamento_id IS NULL;
3
4 -- Empregados que ja tem email cadastrado
5 SELECT * FROM empregados WHERE email IS NOT NULL;
```

Erro clássico

WHERE departamento_id = NULL **nunca retorna nada**, mesmo que existam linhas com NULL. Use sempre IS NULL.

DISTINCT e Aliases (AS)

DISTINCT elimina linhas duplicadas. AS renomeia colunas/tabelas para legibilidade.

```
1 -- Cidades distintas onde temos empregados
2 SELECT DISTINCT cidade FROM empregados;
3
4 -- Aliases em colunas
5 SELECT nome AS funcionario, salario AS remuneracao
6 FROM empregados;
7
8 -- Alias de tabela (vai facilitar JOINS no Modulo 6)
9 SELECT e.nome, e.salario
10 FROM empregados AS e
11 WHERE e.departamento_id = 1;
```


Ordenando com ORDER BY e limitando com LIMIT

```
1 -- Ordem crescente (padrao -- ASC pode ser omitido)
2 SELECT * FROM empregados ORDER BY nome;
3
4 -- Ordem decrescente
5 SELECT * FROM empregados ORDER BY salario DESC;
6
7 -- Multiplas colunas: primeiro por dept, depois por nome
8 SELECT * FROM empregados
9 ORDER BY departamento_id ASC, nome ASC;
10
11 -- Limitar 5 primeiras linhas
12 SELECT * FROM empregados ORDER BY salario DESC LIMIT 5;
13
14 -- Pular 5 e pegar as proximas 10 (paginacao)
15 SELECT * FROM empregados
16 ORDER BY id LIMIT 10 OFFSET 5;
```

Resolva no MySQL

Use as tabelas departamentos e empregados populadas no Lab 2.

- 1 Liste empregados com salário entre R\$ 3.000 e R\$ 7.000 (use BETWEEN).
- 2 Liste empregados cujos nomes começam com “M” ou “J”.
- 3 Liste empregados que **não** estão nos departamentos 2, 3 ou 5.
- 4 Liste os 3 empregados mais bem pagos da empresa.
- 5 Mostre as cidades distintas em que a empresa tem funcionários.
- 6 Liste empregados *sem* departamento atribuído.
- 7 Liste empregados ordenados por departamento crescente e, dentro de cada departamento, por salário decrescente.

Por que dominar SQL é estratégico?

Você Sabia?

Segundo o LinkedIn, **SQL é a habilidade técnica mais demandada** em vagas de tecnologia e dados desde 2020 LinkedIn Talent Blog – Most In-Demand Skills. Profissionais que dominam SQL intermediário ganham em média **30% a mais** do que aqueles com conhecimento apenas básico Glassdoor – SQL Developer Salaries.

Impacto no Dia a Dia

- O **iFood** usa funções agregadas (COUNT, AVG) para gerar relatórios em tempo real sobre milhões de pedidos diários iFood Tech – Engineering Blog.
- Analistas de dados no **Magazine Luiza** utilizam GROUP BY com HAVING para identificar padrões de compra e otimizar estoques Luizalabs – Engineering Blog.
- Uma consulta SQL bem otimizada com WHERE indexado pode executar em **milissegundos** o que uma busca linear levaria **minutos** MySQL 8.0 Docs – How MySQL Uses Indexes.

- Funções agregadas: COUNT, SUM, AVG, MIN, MAX.
- Agrupamento com GROUP BY.
- Filtros pós-agregação com HAVING (e a diferença em relação a WHERE).
- Introdução a **subconsultas básicas** (subqueries no WHERE e no FROM).

Funções agregadas realizam cálculos sobre um *conjunto* de linhas e retornam *um* valor.

- COUNT(*) – conta linhas (incluindo as com NULL).
- COUNT(coluna) – conta valores *não-NULL* de uma coluna.
- SUM(coluna) – soma valores numéricos.
- AVG(coluna) – média aritmética.
- MIN(coluna) / MAX(coluna) – menor / maior valor.

```
1 -- Total de empregados
2 SELECT COUNT(*) AS total FROM empregados;
3
4 -- Folha salarial total e media
5 SELECT SUM(salario) AS folha_total,
6        AVG(salario) AS salario_medio
7 FROM empregados;
8
9 -- Maior e menor salario
10 SELECT MAX(salario) AS maior, MIN(salario) AS menor
11 FROM empregados;
```

COUNT(*) vs COUNT(coluna) – a pegadinha do NULL

```
1 -- Suponha 10 empregados, mas 2 com email = NULL
2 SELECT COUNT(*) FROM empregados;           -- retorna 10
3 SELECT COUNT(email) FROM empregados;       -- retorna 8
4 SELECT COUNT(DISTINCT cidade) FROM empregados; -- valores unicos
```

Regra prática

- Quer contar **linhas**? Use COUNT(*).
- Quer contar **quantos valores existem** numa coluna (ignorando NULLs)? Use COUNT(coluna).
- Quer contar **valores diferentes**? Use COUNT(DISTINCT coluna).

GROUP BY agrupa as linhas por **valores iguais** de uma ou mais colunas. Funções agregadas passam a operar *por grupo*.

```
1 -- Quantos empregados em cada departamento
2 SELECT departamento_id, COUNT(*) AS total
3 FROM empregados
4 GROUP BY departamento_id;
5
6 -- Salario medio por departamento
7 SELECT departamento_id, AVG(salario) AS salario_medio
8 FROM empregados
9 GROUP BY departamento_id;
10
11 -- Mais agregacoes simultaneas no mesmo grupo
12 SELECT departamento_id,
13        COUNT(*) AS qtd,
14        SUM(salario) AS folha,
15        MIN(salario) AS menor,
16        MAX(salario) AS maior
17 FROM empregados
18 GROUP BY departamento_id;
```

A regra que cai em prova

Toda coluna no SELECT que **não está** dentro de uma função agregada **deve** aparecer no GROUP BY.

```
1 -- ERRADO (vai falhar em ONLY_FULL_GROUP_BY do MySQL 8)
2 SELECT departamento_id, nome, COUNT(*)
3 FROM empregados
4 GROUP BY departamento_id;
5
6 -- CERTO
7 SELECT departamento_id, nome, COUNT(*)
8 FROM empregados
9 GROUP BY departamento_id, nome;
```

Por quê?

Cada linha do resultado representa um *grupo*. Se você pedisse nome dentro do grupo “departamento_id = 1”, o SGBD não saberia qual escolher — pode haver vários empregados nesse departamento.

HAVING – filtrando depois do agrupamento

WHERE filtra *linhas individuais antes* do agrupamento.

HAVING filtra *grupos resultantes depois* do agrupamento.

```
1 -- Departamentos com mais de 5 empregados
2 SELECT departamento_id, COUNT(*) AS qtd
3 FROM empregados
4 GROUP BY departamento_id
5 HAVING COUNT(*) > 5;
6
7 -- Departamentos cuja folha salarial passa de 30 mil
8 SELECT departamento_id, SUM(salario) AS folha
9 FROM empregados
10 GROUP BY departamento_id
11 HAVING SUM(salario) > 30000;
```

WHERE x HAVING – combinando

Você pode usar os dois na mesma consulta. WHERE executa primeiro:

```
1 SELECT departamento_id, AVG(salario) AS media
2 FROM empregados
3 WHERE data_admissao >= '2023-01-01'      -- filtra linhas
4 GROUP BY departamento_id
5 HAVING AVG(salario) > 4000;              -- filtra grupos
```

Embora a gente *escreva* SELECT primeiro, o SGBD *executa* as cláusulas nesta ordem. Memorizar isto resolve 80% das dúvidas em SQL:

- 1 FROM – de qual tabela vem o dado.
- 2 WHERE – filtra linhas individuais.
- 3 GROUP BY – forma os grupos.
- 4 HAVING – filtra grupos.
- 5 SELECT – escolhe colunas / aplica agregações.
- 6 ORDER BY – ordena o resultado.
- 7 LIMIT – corta o número de linhas.

Por que isto importa?

Por isso HAVING pode usar COUNT(*) mas WHERE não pode: quando WHERE executa, os grupos ainda não existem.

Uma **subconsulta** é um `SELECT` *dentro* de outro comando SQL. Em BD I focamos em três usos básicos:

- 1 Subconsulta no `WHERE` retornando **um único valor**.
- 2 Subconsulta no `WHERE` com `IN` retornando **uma lista**.
- 3 Subconsulta no `FROM` (**tabela derivada**).

Boas práticas

Subconsultas tornam SQL mais legível, mas **podem ser lentas** em tabelas grandes. Quando possível, prefira `JOIN` (Módulo 6). Otimização vai ser detalhada lá.

```
1 -- Empregados que ganham mais que a media da empresa
2 SELECT nome, salario
3 FROM empregados
4 WHERE salario > (SELECT AVG(salario) FROM empregados);
5
6 -- Empregado com o maior salario
7 SELECT nome, salario
8 FROM empregados
9 WHERE salario = (SELECT MAX(salario) FROM empregados);
```

Cuidado!

A subconsulta com = **deve** retornar *exatamente uma linha com uma coluna*. Se retornar mais de uma, o MySQL aborta com erro “*Subquery returns more than 1 row*”. Para múltiplos valores, use IN.

```
1 -- Empregados dos departamentos que tem mais de 5 pessoas
2 SELECT nome
3 FROM empregados
4 WHERE departamento_id IN (
5     SELECT departamento_id
6     FROM empregados
7     GROUP BY departamento_id
8     HAVING COUNT(*) > 5
9 );
10
11 -- Empregados que NAO trabalham em departamentos da sede de Recife
12 SELECT nome
13 FROM empregados
14 WHERE departamento_id NOT IN (
15     SELECT id FROM departamentos WHERE cidade = 'Recife'
16 );
```

Subconsulta no FROM (tabela derivada)

A subconsulta funciona como uma *tabela temporária* dentro do FROM:

```
1 -- Considere uma "tabela" das medias salariais por departamento.  
2 -- Depois, filtramos so os departamentos com media acima de 4000.  
3 SELECT *  
4 FROM (  
5     SELECT departamento_id,  
6            AVG(salario) AS media  
7     FROM empregados  
8     GROUP BY departamento_id  
9 ) AS resumo  
10 WHERE resumo.media > 4000  
11 ORDER BY resumo.media DESC;
```

Exigência do MySQL

Toda tabela derivada precisa de um **alias** obrigatório (no exemplo, resumo).

Resolva todas no MySQL

- 1 Conte quantos empregados existem em cada departamento.
- 2 Liste a folha salarial total da empresa.
- 3 Liste *somente* os departamentos com folha salarial superior a R\$ 20.000.
- 4 Liste o nome do(s) empregado(s) com o *maior* salário da empresa.
- 5 Liste empregados que ganham acima da média do seu departamento (use subquery).
- 6 Liste os 3 departamentos com mais empregados (GROUP BY + ORDER BY + LIMIT).
- 7 Encontre departamentos cuja média salarial está acima da média geral da empresa.

Simule um relatório executivo

Você é o analista de dados da empresa. Entregue ao gestor:

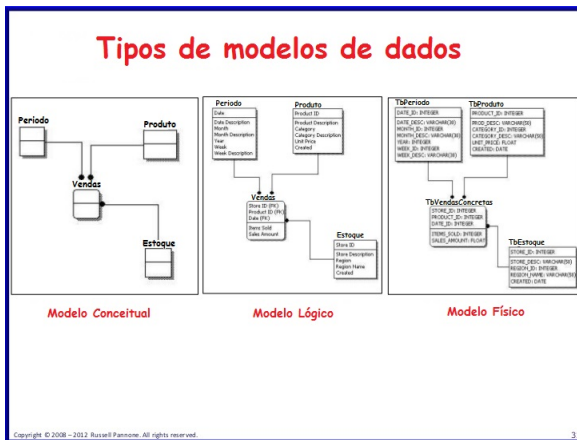
- 1 Tabela com **nome do departamento**, **quantidade de funcionários**, **folha total**, **salário médio**, **maior salário** e **menor salário**.
- 2 Lista de departamentos cuja média salarial é *superior* à média geral da empresa.
- 3 Top 5 funcionários mais bem pagos.
- 4 Quantidade de funcionários sem departamento (IS NULL).

Entrega: arquivo `relatorio.sql` com as 4 consultas comentadas e o output exportado em CSV pelo Workbench.

- Revisar os conceitos do **Modelo Entidade–Relacionamento (MER)** e do **DER**.
- Compreender a diferença entre modelo **conceitual**, **lógico** e **físico**.
- Aplicar regras práticas de **mapeamento DER** → **tabelas**.
- Tratar relacionamentos 1:1, 1:N e N:N em SQL.

O que é modelagem de dados?

- **Definição:** a modelagem de dados é o processo de criar um modelo visual das informações que serão armazenadas.
- **Objetivo:** estruturar os dados para que possam ser armazenados, acessados e gerenciados de maneira eficiente.



Fluxo profissional

- ❶ **Modelo Conceitual** (DER/MER): *o que* existe? Comunicação com clientes e analistas. Independente de tecnologia.
- ❷ **Modelo Lógico**: *como* se organiza? Tabelas, chaves, FKs e tipos genéricos. Ainda independente de SGBD.
- ❸ **Modelo Físico**: *onde e com qual tecnologia?* Já considera InnoDB, índices, charsets, partições etc. (foco do MySQL).

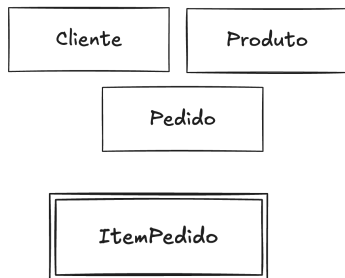
Dica do mercado

Empresas como TOTVS e Stefanini exigem que toda modelagem passe pelas três fases antes da implementação. Pular etapas é a causa número 1 de retrabalho em projetos de banco de dados TOTVS Blog – Modelagem de Dados.

Existem várias formas de desenhar um Modelo Entidade-Relacionamento. Em sala vamos usar principalmente:

- **Notação de Peter Chen:** clássica, didática, usa retângulos, losangos e elipses. Muito boa para introdução.
- **Notação de James Martin (“pé-de-galinha”):** compacta, adotada por ferramentas como MySQL Workbench, brModelo, dbdiagram.io.
- **Notação de Heuser** (Chen adaptada): popular em livros brasileiros.

A notação de Peter Chen é caracterizada por sua simplicidade. As entidades representam objetos ou conceitos do mundo real que possuem existência própria no contexto do sistema.



Entidades: representam objetos ou conceitos do mundo real que possuem existência própria. Ex.: Cliente, Produto, Funcionário.

Entidade fraca: não possui um identificador único próprio e depende de uma entidade forte. Ex.: Item de Pedido (depende do Pedido), Dependente (depende do Funcionário).

Simples: representam características ou propriedades das entidades.



Chave primária: identifica unicamente cada entidade dentro de um conjunto.



A **cardinalidade** indica quantas instâncias de uma entidade podem se relacionar com instâncias de outra. As três principais são:

- **1:1** – um para um.
- **1:N** – um para muitos.
- **N:N** – muitos para muitos.

Exemplos do dia a dia

- Pessoa ↔ Passaporte: **1:1**.
- Cliente ↔ Pedido: **1:N** (1 cliente faz vários pedidos).
- Aluno ↔ Disciplina: **N:N** (vários alunos cursam várias disciplinas).

Cada registro em A se relaciona com no máximo um registro em B.



Notação 1:1: para cada instância de uma entidade (uma pessoa) há uma e somente uma instância associada na outra (passaporte).

Um registro de A se relaciona com vários registros de B; cada registro de B se relaciona com apenas um de A.



Notação 1:N: uma instância da entidade A pode se relacionar com muitas instâncias da entidade B.

Vários registros de A se relacionam com vários registros de B. Geralmente é resolvida com uma **tabela associativa**.



Notação N:N: muitas instâncias de A podem estar associadas a muitas instâncias de B.

Entidades:

- Representadas por retângulos. Exemplo: Clientes, Compra.

Atributos:

- Representados por elipses. Exemplo: Nome, Código.

Relacionamentos:

- Representados por losangos. Exemplo: Realiza, Contém, Faz.

Chave primária:

- Um atributo ou conjunto de atributos que identifica de forma única uma entidade.

- É a **representação intermediária** entre o conceitual (DER) e o físico (CREATE TABLE).
- Já fala em **tabelas, colunas, chaves primárias, chaves estrangeiras**, mas ainda independe do SGBD.
- **Características:**
 - Cada entidade → uma tabela.
 - Cada atributo → uma coluna.
 - Relacionamentos → representados por chaves estrangeiras (FKs) ou por tabelas associativas.
 - Aplicação de normalização (Módulo 7).

As 4 regras de ouro

- 1 **Entidade forte** → vira uma tabela; o atributo identificador vira a **PK**.
- 2 **Atributos simples** → viram colunas com tipo apropriado.
- 3 **Relacionamento 1:N** → a FK vai para o lado “N”.
- 4 **Relacionamento N:N** → vira uma **tabela associativa**, com FKs para as duas tabelas e PK composta.

Casos especiais

- **1:1**: a FK vai para o lado mais “dependente”. Pode-se também unir as duas entidades em uma só tabela.
- **Entidade fraca**: vira tabela cuja PK *inclui* a FK da entidade forte.
- **Atributo multivalorado**: vira uma tabela à parte (ex.: “telefones do cliente”).
- **Atributo composto**: cada componente vira uma coluna (ex.: endereço → rua, número, bairro, cidade).

Conceitual: Cliente (1) – Faz – (N) Pedido.

```
1 CREATE TABLE clientes (  
2     id      INT AUTO_INCREMENT PRIMARY KEY,  
3     nome    VARCHAR(100) NOT NULL,  
4     email   VARCHAR(120) UNIQUE  
5 );  
6  
7 CREATE TABLE pedidos (  
8     id          INT AUTO_INCREMENT PRIMARY KEY,  
9     data_pedido DATE NOT NULL,  
10    total       DECIMAL(10,2) DEFAULT 0,  
11    cliente_id  INT NOT NULL,  
12    FOREIGN KEY (cliente_id) REFERENCES clientes(id)  
13 );
```

Regra aplicada: a FK cliente_id vai para a tabela do lado “N” (pedidos).

Conceitual: Aluno (N) – Cursa – (N) Disciplina.

```
1 CREATE TABLE alunos (  
2     id     INT AUTO_INCREMENT PRIMARY KEY,  
3     nome  VARCHAR(100) NOT NULL  
4 );  
5  
6 CREATE TABLE disciplinas (  
7     id     INT AUTO_INCREMENT PRIMARY KEY,  
8     nome  VARCHAR(100) NOT NULL  
9 );  
10  
11 -- Tabela associativa  
12 CREATE TABLE matriculas (  
13     aluno_id      INT NOT NULL,  
14     disciplina_id INT NOT NULL,  
15     semestre      VARCHAR(7) NOT NULL,      -- ex: 2026.1  
16     nota_final     DECIMAL(4,2),  
17     PRIMARY KEY (aluno_id, disciplina_id, semestre),  
18     FOREIGN KEY (aluno_id) REFERENCES alunos(id),  
19     FOREIGN KEY (disciplina_id) REFERENCES disciplinas(id)  
20 );
```

Conceitual: Pessoa (1) – Possui – (1) Passaporte.

```
1 CREATE TABLE pessoas (  
2     id     INT AUTO_INCREMENT PRIMARY KEY,  
3     nome  VARCHAR(100) NOT NULL,  
4     cpf   VARCHAR(14) UNIQUE  
5 );  
6  
7 CREATE TABLE passaportes (  
8     id          INT AUTO_INCREMENT PRIMARY KEY,  
9     numero      VARCHAR(20) UNIQUE NOT NULL,  
10    validade    DATE,  
11    pessoa_id   INT UNIQUE NOT NULL,    -- UNIQUE garante 1:1  
12    FOREIGN KEY (pessoa_id) REFERENCES pessoas(id)  
13 );
```

Truque: a FK `pessoa_id` é `UNIQUE` — isso impede que uma pessoa tenha mais de um passaporte (transformando a relação em 1:1).

Você Sabia?

Segundo pesquisa da IBM, **70% dos problemas em sistemas** de banco de dados têm origem em uma **modelagem conceitual mal feita** IBM – What is Data Modeling. Corrigir erros de modelagem após a implementação custa até **100 vezes mais** do que corrigi-los na fase de projeto IBM – What is Data Modeling.

Casos reais

- O **SUS** utiliza diagramas ER para modelar mais de **200 entidades** que gerenciam dados de 210 milhões de brasileiros DATASUS.
- A **Amazon** redesenhou sua modelagem em 2015, com **40% de redução no tempo de consulta** do catálogo AWS Database Blog.
- A ferramenta brasileira **brModelo** (Prof. Heuser) é usada em mais de **500 instituições** Heuser, 2009.

Modele e implemente

Para cada cenário abaixo, desenhe o DER (papel ou ferramenta) e escreva os CREATE TABLE correspondentes:

- ❶ **Biblioteca:** Livros, Autores e Empréstimos. Um livro tem vários autores e vice-versa. Cada empréstimo é feito por um aluno.
- ❷ **Hospital:** Médicos, Pacientes e Consultas. Um médico atende várias consultas, cada consulta tem um paciente.
- ❸ **Streaming:** Usuários, Filmes e Avaliações (com nota de 0 a 10).
- ❹ **Loja virtual:** Cliente, Pedido, Produto, Item de Pedido (entidade fraca relacionada ao Pedido).

Atividade de Modelagem – 60 minutos

Em duplas, escolham um dos cenários acima e:

- 1 Desenhem o DER no brModelo ou no MySQL Workbench (modo EER).
- 2 Identifiquem as cardinalidades.
- 3 Apliquem as 4 regras de mapeamento.
- 4 Gerem o script .sql CREATE TABLE (com PK, FK e constraints).
- 5 Insiram pelo menos 5 linhas em cada tabela.
- 6 Façam um SELECT simples em cada tabela para validar.

Entrega: arquivo PDF com o DER + arquivo .sql executável.

- Compreender por que precisamos de **JOINS** num modelo relacional normalizado.
- Visualizar as junções com **Diagramas de Venn**.
- Dominar a sintaxe e o comportamento de **INNER JOIN**, **LEFT JOIN**, **RIGHT JOIN** e **CROSS JOIN**.
- Praticar **JOINS** combinados com **WHERE**, **GROUP BY** e funções agregadas.

Por que precisamos de JOINS?

Em um banco normalizado, **dados relacionados ficam separados em tabelas distintas** (Cliente, Pedido, Produto...). **JOIN** é o mecanismo para *recompôr* essas relações em uma única consulta.

Analogia

Pense em duas listas de papel: uma com nomes de alunos e outra com matrículas em disciplinas. Para descobrir *quais disciplinas cada aluno cursa*, você precisa cruzar as duas listas pelo identificador do aluno — JOIN faz exatamente isso de forma automática e otimizada.

Sintaxe geral

```
1 SELECT  colunas
2 FROM    tabelaA [alias_a]
3 JOIN    tabelaB [alias_b] ON alias_a.fk = alias_b.pk;
```

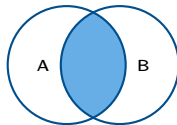
Esquema de exemplo (vamos usar nas próximas slides)

```
1 CREATE TABLE departamentos (  
2     id      INT PRIMARY KEY,  
3     nome    VARCHAR(50)  
4 );  
5  
6 CREATE TABLE empregados (  
7     id              INT PRIMARY KEY,  
8     nome            VARCHAR(50),  
9     departamento_id INT,          -- pode ser NULL!  
10    FOREIGN KEY (departamento_id) REFERENCES departamentos(id)  
11 );  
12  
13 INSERT INTO departamentos VALUES  
14 (1, 'TI'), (2, 'RH'), (3, 'Financeiro'), (4, 'Marketing');  
15  
16 INSERT INTO empregados VALUES  
17 (10, 'Ana',      1),  
18 (11, 'Bruno',    1),  
19 (12, 'Carla',    2),  
20 (13, 'Daniel',   3),  
21 (14, 'Erica',    NULL); -- empregado sem departamento  
22 -- Note: o departamento 4 nao tem ninguem!
```

Os JOINS em Diagramas de Venn

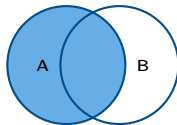
Cada conjunto representa as linhas de uma tabela. A interseção é o “*match*” das chaves.

INNER JOIN



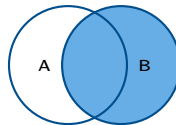
só linhas com match

LEFT JOIN



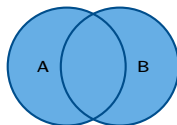
tudo de A + match de B

RIGHT JOIN



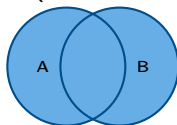
tudo de B + match de A

CROSS JOIN



todas as combinações: $|A| \times |B|$

(FULL OUTER)



nao existe
nativo no MySQL

Curiosidade

O MySQL não tem FULL OUTER JOIN nativamente. Quando precisamos dele, emulamos com LEFT JOIN UNION RIGHT JOIN (assunto avançado).

INNER JOIN – só o que combina dos dois lados

Retorna apenas as linhas que possuem correspondência em ambas as tabelas.



```
1 SELECT e.nome AS empregado,  
2       d.nome AS departamento  
3 FROM empregados e  
4 INNER JOIN departamentos d ON e.departamento_id = d.id;
```


empregado	departamento
Ana	TI
Bruno	TI
Carla	RH
Daniel	Financeiro

O que sumiu?

- **Erica** (departamento_id = NULL) não apareceu.
- **Marketing** (sem empregados) também não apareceu.

INNER JOIN é *intolerante a ausências dos dois lados*.

Retorna todas as linhas da tabela à esquerda, com NULL onde não houver correspondência à direita.



```
1 SELECT e.nome AS empregado ,  
2       d.nome AS departamento  
3 FROM empregados e  
4 LEFT JOIN departamentos d ON e.departamento_id = d.id;
```

empregado	departamento
Ana	TI
Bruno	TI
Carla	RH
Daniel	Financeiro
Erica	NULL

Use o LEFT JOIN quando...

... você quiser todas as linhas da tabela “principal”, mesmo as órfãs. Ex.: *“liste todos os empregados, mostrando o departamento se houver”*.

Truque: encontrar órfãos com LEFT JOIN

Como achar empregados **sem** departamento? Filtre o LEFT JOIN por NULL:

```
1 SELECT e.nome AS empregado
2 FROM empregados e
3 LEFT JOIN departamentos d ON e.departamento_id = d.id
4 WHERE d.id IS NULL;
```

Resultado: apenas Erica. Esse é um padrão muito usado em auditoria de dados.

Retorna todas as linhas da tabela à direita, com NULL onde não houver correspondência à esquerda.



```
1 SELECT e.nome AS empregado ,  
2       d.nome AS departamento  
3 FROM empregados e  
4 RIGHT JOIN departamentos d ON e.departamento_id = d.id;
```

empregado	departamento
Ana	TI
Bruno	TI
Carla	RH
Daniel	Financeiro
NULL	Marketing

Você sabia?

Todo RIGHT JOIN **pode ser reescrito como** LEFT JOIN trocando a ordem das tabelas. Por isso, na prática, equipes de SQL costumam **padronizar tudo em** LEFT JOIN para facilitar a leitura.

```
1 -- RIGHT JOIN
2 SELECT e.nome, d.nome
3 FROM empregados e
4 RIGHT JOIN departamentos d ON e.departamento_id = d.id;
5
6 -- ...e o LEFT JOIN equivalente (so trocamos a ordem)
7 SELECT e.nome, d.nome
8 FROM departamentos d
9 LEFT JOIN empregados e ON e.departamento_id = d.id;
```

Convenção do mercado

A equipe Stack Overflow analisou milhões de queries SQL e identificou que mais de **95% dos JOINS externos** usam LEFT JOIN, mesmo quando o autor poderia escolher RIGHT. Isto deixa o código mais previsível.

Combina **cada linha de A com cada linha de B**. Sem cláusula ON (não há condição de junção).

- Resultado tem $|A| \times |B|$ linhas.
- Útil para gerar grades, calendários, combinações de teste.
- **Cuidado:** $1\,000 \times 1\,000 = 1\,000\,000$ linhas!

```
1 -- Todas as combinacoes de empregados x departamentos
2 SELECT e.nome AS empregado, d.nome AS departamento
3 FROM empregados e
4 CROSS JOIN departamentos d;
5
6 -- Equivalente, sintaxe antiga (nao recomendada)
7 SELECT e.nome, d.nome
8 FROM empregados e, departamentos d;
```


Cenário: gerar um *relatório de vendas projetado* para todas as combinações “Produto × Mês de 2026”, mesmo as combinações sem venda.

```
1  -- Tabela auxiliar com 12 meses
2  CREATE TABLE meses (mes_num INT PRIMARY KEY, mes_nome VARCHAR(15));
3  INSERT INTO meses VALUES
4  (1, 'Jan'), (2, 'Fev'), (3, 'Mar'), (4, 'Abr'), (5, 'Mai'), (6, 'Jun'),
5  (7, 'Jul'), (8, 'Ago'), (9, 'Set'), (10, 'Out'), (11, 'Nov'), (12, 'Dez');
6
7  -- Grade Produto x Mes
8  SELECT p.nome AS produto, m.mes_nome AS mes
9  FROM produtos p
10 CROSS JOIN meses m
11 ORDER BY p.nome, m.mes_num;
```

Use CROSS JOIN com critério

Em produção, use sempre que precisar de combinatória *intencional*. Se chegou aí “por acidente” (esqueceu de ON), você criou um produto cartesiano que pode estourar a memória do servidor.

Tipo	Retorna	Quando usar
INNER JOIN	só linhas que têm match dos dois lados	junção “segura”, sem NULL
LEFT JOIN	todas da esquerda + match da direita	“mostre tudo da principal”
RIGHT JOIN	todas da direita + match da esquerda	espelho do LEFT (raro)
CROSS JOIN	produto cartesiano $ A \times B $	grades, combinações

Mantra do JOIN

- 1 Sempre dê **alias**es curtos (e, d, p).
- 2 Sempre use ON (não confunda com WHERE).
- 3 LEFT JOIN é o “primo amigável”. Use-o por padrão em junções externas.
- 4 Cuidado com NULL no lado da junção — afeta filtros posteriores.

JOIN com 3 tabelas (caso clássico)

Cliente → Pedido → Item de Pedido → Produto.

```
1 SELECT c.nome      AS cliente ,
2        p.id        AS pedido ,
3        prod.nome    AS produto ,
4        ip.qtd       AS quantidade ,
5        prod.preco * ip.qtd AS subtotal
6 FROM clientes c
7 INNER JOIN pedidos      p      ON p.cliente_id = c.id
8 INNER JOIN itens_pedido ip     ON ip.pedido_id  = p.id
9 INNER JOIN produtos     prod   ON prod.id       = ip.produto_id
10 ORDER BY c.nome, p.id;
```

Lê-se assim

“Para cada cliente, para cada pedido dele, para cada item desse pedido, mostre o produto e o subtotal.”

Total faturado por cliente em 2026:

```
1 SELECT c.nome AS cliente,  
2       SUM(prod.preco * ip.qtd) AS total_gasto  
3 FROM clientes c  
4 INNER JOIN pedidos p      ON p.cliente_id = c.id  
5 INNER JOIN itens_pedido ip ON ip.pedido_id = p.id  
6 INNER JOIN produtos prod  ON prod.id      = ip.produto_id  
7 WHERE YEAR(p.data_pedido) = 2026  
8 GROUP BY c.id, c.nome  
9 ORDER BY total_gasto DESC;
```

Truque do agrupamento

Sempre agrupe por `c.id` (a PK), e *também* pode incluir `c.nome` no GROUP BY para satisfazer o ONLY_FULL_GROUP_BY do MySQL 8.

LEFT JOIN + agregação (cuidado!)

Liste todos os clientes e quanto cada um gastou (mesmo quem nunca comprou).

```
1 SELECT c.nome,  
2        COALESCE(SUM(prod.preco * ip.qtd), 0) AS total_gasto  
3 FROM clientes c  
4 LEFT JOIN pedidos p      ON p.cliente_id = c.id  
5 LEFT JOIN itens_pedido ip ON ip.pedido_id = p.id  
6 LEFT JOIN produtos prod  ON prod.id      = ip.produto_id  
7 GROUP BY c.id, c.nome  
8 ORDER BY total_gasto DESC;
```

Por que COALESCE?

Para clientes sem pedidos, SUM(...) retorna NULL. COALESCE(SUM(...), 0) substitui NULL por zero — a apresentação fica *muito* mais limpa.

Use o esquema empregados/departamentos populado anteriormente

- 1 Liste empregados com o nome do seu departamento (INNER JOIN).
- 2 Liste todos os empregados, mostrando NULL no departamento se for o caso (LEFT JOIN).
- 3 Liste todos os departamentos, com NULL no nome do empregado se o departamento estiver vazio (RIGHT JOIN ou LEFT JOIN invertido).
- 4 Liste empregados *sem* departamento (LEFT JOIN ... WHERE d.id IS NULL).
- 5 Liste departamentos *sem* empregados.
- 6 Quantos empregados cada departamento tem? (Inclua zero para os vazios.)
- 7 Liste todas as combinações empregado-departamento usando CROSS JOIN e explique por que esta consulta normalmente não é o que queremos.

Atividade da Aula – 90 minutos (em duplas)

A partir das tabelas `clientes`, `pedidos`, `produtos` e `itens_pedido`:

- 1 Crie a tabela `itens_pedido`(`pedido_id`, `produto_id`, `qtd`, PK composta) e popule com pelo menos 15 itens distribuídos em diversos pedidos.
- 2 Liste o *detalhe* de cada pedido (cliente, produto, qtd, subtotal).
- 3 Liste o *total* faturado por cliente em 2026.
- 4 Liste *clientes que ainda não fizeram nenhum pedido* (LEFT JOIN + IS NULL).
- 5 Liste os **3 produtos mais vendidos** (em quantidade total).
- 6 Liste o **ticket médio** (total/pedido) por cliente.
- 7 Liste produtos *nunca vendidos*.

Entrega: arquivo `lab_vendas.sql` com cada consulta comentada e screenshots dos resultados.

Você Sabia?

- Cada consulta de uma tela do **iFood** acessa em média **6 a 8 tabelas via JOINS** iFood Tech – Engineering Blog.
- Em sistemas bancários como o **Bradesco**, JOINS mal escritos foram causa de quedas de performance documentadas em 2021 MySQL 8.0 Docs.
- Saber usar LEFT JOIN ... IS NULL para encontrar dados “órfãos” é uma das técnicas mais cobradas em **entrevistas técnicas** de Data Engineer.

- Compreender o que são **anomalias** em tabelas mal projetadas.
- Aplicar a **1ª, 2ª e 3ª Forma Normal** (foco prático em BD I).
- Reconhecer quando uma tabela já está adequada.
- Saber explicar a normalização para um colega não-DBA.

Normalização é o processo sistemático de organizar tabelas de um banco de dados relacional para **reduzir redundâncias** e **eliminar anomalias** de inserção, atualização e exclusão Date, 2004.

Anomalias causadas pela falta de normalização

- **Inserção:** impossível inserir um dado sem outro não relacionado.
- **Atualização:** alterar um dado exige atualização em múltiplos lugares.
- **Exclusão:** excluir um registro apaga informações que deveriam persistir.

As Formas Normais foram propostas por **E. F. Codd** (1970–1971) e refinadas posteriormente Codd, 1970 Date, 2004.

Cada Forma Normal é como um **nível num jogo**: resolver cada anomalia nos aproxima do *tesouro* — um banco de dados limpo, consistente e eficiente Heuser, 2009!



Tabela “denormalizada” – nosso ponto de partida

Imagine uma tabela única de vendas:

```
1 CREATE TABLE vendas_ruim (  
2     pedido_id    INT,  
3     cliente_id   INT,  
4     cliente_nome VARCHAR(100),  
5     cliente_cidade VARCHAR(50),  
6     cliente_uf   CHAR(2),  
7     produtos     VARCHAR(500),  -- "Caneta, Caderno, Lapis" !!!  
8     data_pedido  DATE,  
9     cidade_regiao VARCHAR(50)   -- "Recife" -> "Nordeste"  
0 );
```

O que está errado aqui?

- produtos viola **1FN** (lista numa célula).
- cliente_* viola **2FN** (depende parcialmente da chave).
- cidade_regiao viola **3FN** (depende de outra coluna não-chave).

Vamos consertar.

1ª Forma Normal (1FN)

Regra: cada coluna deve conter **valores atômicos** (não divisíveis); proibido grupos repetidos.

Ferindo a 1FN

```
1 -- Tabela "Pedido" com lista no campo "Itens"
2 | pedido_id | itens |
3 |-----|-----|
4 | 1 | Caneta, Caderno, Lapis |
5 | 2 | Borracha, Caneta |
```

Solução: criar a tabela pedido_item, com cada produto em uma linha:

```
1 CREATE TABLE pedido_item (
2     pedido_id INT,
3     produto VARCHAR(50),
4     PRIMARY KEY (pedido_id, produto)
5 );
6 INSERT INTO pedido_item VALUES
7 (1, 'Caneta'), (1, 'Caderno'), (1, 'Lapis'),
8 (2, 'Borracha'), (2, 'Caneta');
```

Listas em uma célula são uma armadilha

Armazenar múltiplos valores separados por vírgula em uma coluna ("Caneta, Caderno, Lápis") viola a 1FN e dificulta consultas, filtros e integridade referencial Date, 2004. Tente responder "*quantos pedidos incluíram caneta?*" com a estrutura errada — só dá errado.

Pergunta para refletir

Como você responderia, em SQL, "*quantos pedidos incluíram caneta?*" nas duas estruturas? Compare a complexidade.

2ª Forma Normal (2FN)

Pré-requisito: estar em 1FN.

Regra: todos os atributos não-chave devem depender *totalmente* da chave primária. Aplicável quando a PK é composta.

Ferindo a 2FN

```
1 -- PK composta: (pedido_id, produto_id)
2 | pedido_id | produto_id | qtd | preco_unit | nome_produto |
```

- `preco_unit` e `nome_produto` dependem **só de** `produto_id`, não da PK toda.
- **Solução:** mover essas colunas para a tabela `produtos`, deixando em `itens_pedido` apenas (`pedido_id`, `produto_id`, `qtd`).

Regra prática

Se um atributo não-chave depende *apenas de parte* da chave composta, há violação de 2FN. Separe em outra tabela Wikipédia – Second Normal Form (2NF).

3ª Forma Normal (3FN)

Pré-requisito: estar em 2FN.

Regra: *nenhum atributo não-chave depende de outro atributo não-chave* — ou seja, sem *dependências transitivas*.

Ferindo a 3FN

```
1 -- Tabela cliente
2 | id_cliente | nome | cidade | uf | regioao |
3 | 1          | Ana  | Recife | PE | NE       |
4 | 2          | Bia  | Natal  | RN | NE       |
```

- regioao depende de uf, não diretamente do id_cliente. Isto é uma dependência **transitiva**.
- **Solução:** criar tabela ufs(uf, regioao) e deixar só uf no cliente.

```
1 CREATE TABLE ufs (
2     uf      CHAR(2) PRIMARY KEY,
3     regioao VARCHAR(15)
4 );
```


Cola

- **1FN**: sem grupos repetidos — valores atômicos.
- **2FN**: 1FN + sem dependência *parcial* da PK.
- **3FN**: 2FN + sem dependência *transitiva* entre não-chaves.

Mantra do Codd

- Cada atributo deve depender **da chave** (1FN),
- **de toda a chave** (2FN),
- **e somente da chave** (3FN). *So help me Codd.*

Você Sabia?

Bancos de dados **não normalizados** consomem até **60% mais espaço em disco** e apresentam **3× mais anomalias** de inserção, atualização e exclusão. A normalização é a base de todo banco relacional bem projetado Wikipédia – Database Normalization.

Cenário Real

- O sistema do **Detran-PE** sofreu inconsistências por tabelas fora da 2FN, causando duplicação de registros de veículos em 2019.
- A **Netflix** mantém tabelas de catálogo em 3FN, mas usa **desnormalização controlada** em caches para ganho de performance Date, 2004.
- **Desnormalizar sem critério** é erro comum de DBAs juniores — sempre normalize primeiro, depois otimize com critério.

Identifique a forma normal violada

Para cada tabela, diga em qual forma normal está e como normalizá-la:

- ❶ `aluno(matricula, nome, telefones)` – onde `telefones` = "81-9999, 81-8888".
- ❷ `nota(aluno_id, disciplina_id, nota, nome_aluno, nome_disciplina)`.
- ❸ `funcionario(id, nome, cep, rua, bairro, cidade, uf)`.
- ❹ `venda(id_venda, id_produto, qtd, descricao_produto, preco_produto)`.
- ❺ `livro(isbn, titulo, autor1, autor2, autor3, editora, cidade_editora)`.

Exercício de Refatoração – 60 minutos

Você recebeu de um colega a seguinte tabela única (vendas_legacy):

- (pedido_id, data, cliente_id, cliente_nome, cidade, uf, regioao, produtos_lista, valor_total).

Sua missão:

- 1 Aponte por escrito todas as violações de 1FN, 2FN e 3FN.
- 2 Reescreva o esquema em conformidade com a 3FN, em SQL (CREATE TABLEs + FKs).
- 3 Crie um INSERT INTO ... SELECT ... que migre dados do legado para o novo modelo.
- 4 Escreva 3 consultas que tenham o mesmo objetivo nas duas estruturas e compare a clareza/desempenho.

Entrega: arquivo normalizacao.sql e um curto README.md explicando as decisões.

Objetivo desta semana (introdutório)

Entender, em alto nível, **como o MySQL guarda dados em disco**. Não vamos configurar servidores nem afinar parâmetros — isto é tópico de BD II e de *Database Administration* (DBA). Aqui buscamos a **intuição** de:

- por que uma consulta às vezes é rápida e às vezes é lenta;
- por que índices ajudam (próximo módulo);
- por que o INT ocupa menos que o VARCHAR(255).

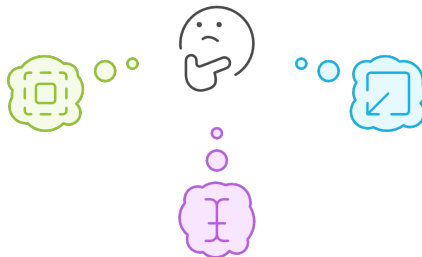
Como os dados são armazenados fisicamente?



Qual é a melhor forma de otimizar o desempenho do banco de dados MySQL?

Aumentar o tamanho da página

Permite armazenar mais registros em uma única página, reduzindo o número de leituras e gravações no disco.



Reduzir o tamanho do bloco

Permite acessar dados mais rapidamente, mas pode aumentar o número de leituras e gravações no disco.

Otimizar o tamanho do registro

Reduz o espaço ocupado por cada registro, permitindo armazenar mais registros em uma única página.

Quando dados são inseridos em uma tabela, o MySQL agrupa as linhas em **páginas**. A página é a menor unidade que o SGBD lê e escreve no disco de uma vez.

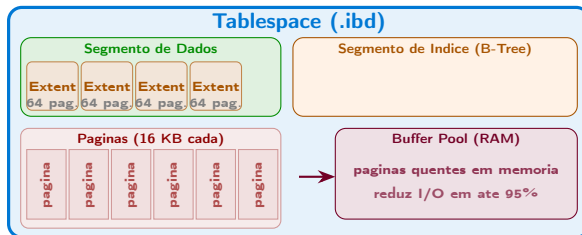
```
1 CREATE TABLE Alunos (  
2     id INT PRIMARY KEY AUTO_INCREMENT,  
3     nome VARCHAR(50),  
4     idade INT  
5 ) ENGINE=InnoDB;  
6  
7 INSERT INTO Alunos (nome, idade)  
8 VALUES ('Joao', 20), ('Maria', 22), ('Lucas', 19), ('Ana', 21);
```


- **Bloco:** agrupamento físico de páginas, conceito mais próximo do SO.
- **Registro:** uma linha da tabela — ocupa um espaço dentro da página.

```
1 INSERT INTO Alunos (nome, idade)
2 VALUES ('Carlos', 23), ('Julia', 20), ('Fernando', 25);
3
4 SELECT * FROM Alunos WHERE idade > 20;
```

- Cada linha recuperada ocupa espaço dentro de uma página.
- Quanto mais registros, mais páginas necessárias — e maior o impacto na organização física.

O mecanismo **InnoDB** organiza dados de forma hierárquica MySQL 8.0 Docs:



Arquitetura InnoDB em Números

- Cada **página** no InnoDB tem tamanho fixo de **16 KB** por padrão MySQL 8.0 Docs.
- Um **extent** agrupa **64 páginas** consecutivas (1 MB).
- O **Buffer Pool** mantém páginas frequentes em RAM, reduzindo I/O em até **95%**.

Você Sabia?

O **Mercado Livre** otimizou sua configuração de páginas InnoDB e reduziu o tempo de resposta de consultas críticas em **50%** apenas ajustando o tamanho do Buffer Pool. Entender a estrutura física é essencial para DBAs — mas em BD I basta a intuição Mercado Livre Tech.

Pense como um DBA junior

- ❶ Por que escolher INT em vez de BIGINT para uma chave primária pequena economiza espaço em *toda* a tabela?
- ❷ Em uma página de 16 KB, cabem aproximadamente quantas linhas se cada linha ocupar 200 bytes?
- ❸ O que acontece (em alto nível) quando você consulta uma página que *não* está no buffer pool?
- ❹ Por que tabelas muito “largas” (com 80 colunas) tendem a ser mais lentas que duas tabelas estreitas em JOIN?

- A indexação é uma técnica para **acelerar o acesso aos dados**, permitindo encontrar registros sem ler a tabela inteira MySQL 8.0 Docs – How MySQL Uses Indexes.
- Assim como o índice de um livro permite localizar capítulos rapidamente, um índice de BD mapeia valores de coluna para a localização física dos registros.

Você Sabia?

Uma tabela com **10 milhões de registros** sem índice pode levar **30 segundos** para uma consulta simples. Com um índice B-tree adequado, a mesma consulta retorna em **menos de 10 ms** — melhoria de **3.000×** Use The Index, Luke.

Quando indexar?

- **Sim:** colunas usadas frequentemente em WHERE, JOIN e ORDER BY.
- **Sim:** chaves estrangeiras que participam de junções.
- **Não:** tabelas muito pequenas (o overhead supera o ganho).
- **Cuidado:** cada índice ocupa espaço e **torna INSERT/UPDATE mais lento** — equilíbrio é a chave MySQL 8.0 Docs.

O que é Índice?

Estrutura de Índice

Valores de Coluna

Os pontos de dados específicos usados para indexar e recuperar informações.



Estrutura de Dados

A estrutura fundamental que permite a organização e recuperação eficiente de dados.

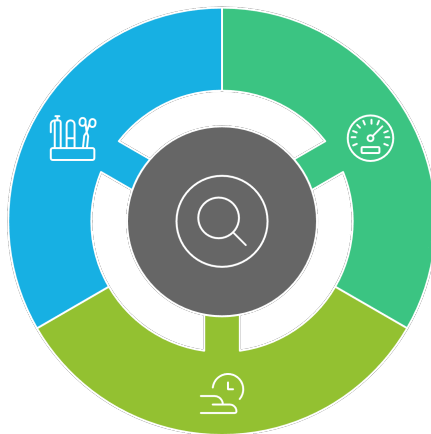
Acesso Rápido

A capacidade de
Banco de Dados I

Benefícios do Uso de Índices

Ordenação

Auxilia na ordenação dos resultados sem sobrecarregar o servidor.



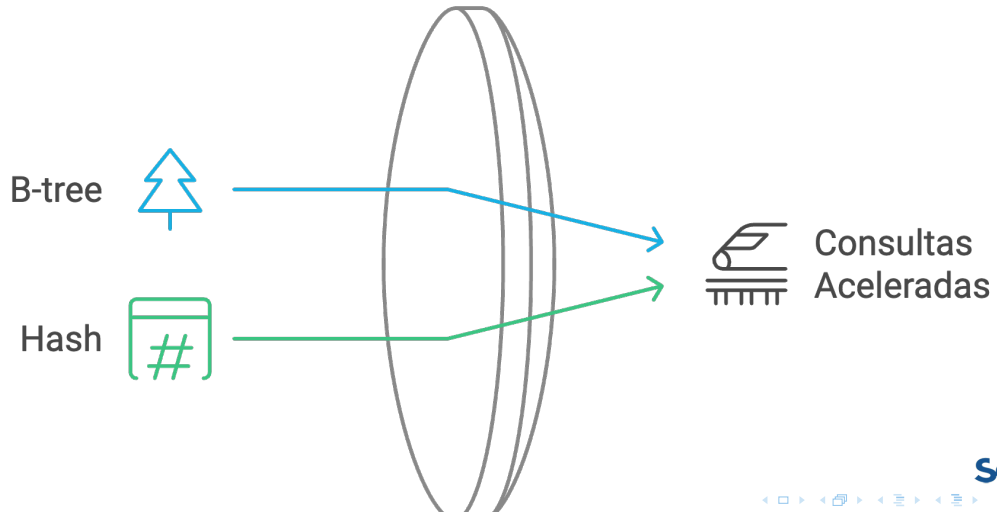
Desempenho

Aumenta a velocidade das operações de busca e filtragem.

Eficiência

Reduz o tempo de
resposta das
Banco de Dados I

Indexação em Bancos de Dados



O índice B-tree organiza os dados em uma árvore balanceada, permitindo busca eficiente em grandes conjuntos de dados MySQL 8.0 Docs.

```
1 CREATE TABLE Produtos (  
2     id      INT PRIMARY KEY AUTO_INCREMENT,  
3     nome    VARCHAR(100),  
4     preco  DECIMAL(10, 2)  
5 ) ENGINE=InnoDB;  
6  
7 INSERT INTO Produtos (nome, preco)  
8 SELECT CONCAT('Produto', FLOOR(RAND() * 1000)), RAND() * 1000  
9 FROM information_schema.tables LIMIT 1000;  
10  
11 -- Consulta sem indice na coluna 'preco'  
12 SELECT * FROM Produtos WHERE preco BETWEEN 100 AND 200;  
13  
14 -- Adicionando um indice B-tree  
15 CREATE INDEX idx_preco ON Produtos(preco);  
16  
17 -- Mesma consulta, agora com indice -- muito mais rapida!  
18 SELECT * FROM Produtos WHERE preco BETWEEN 100 AND 200;
```

O comando EXPLAIN mostra como o MySQL *planeja* executar uma consulta:

```
1 EXPLAIN SELECT * FROM Produtos WHERE preco BETWEEN 100 AND 200;
```

- Se na coluna type aparece ALL, é *full table scan* (ruim).
- Se aparece range ou ref, o índice está sendo usado (bom).
- Coluna key mostra qual índice está sendo aplicado.

Dica Profissional

Em entrevistas técnicas para vagas de Data Engineer, é comum o entrevistador pedir para você “*otimizar esta consulta*”. A primeira ação de qualquer candidato sênior é rodar EXPLAIN.

- Usa função de *hash* para mapear valores a posições.
- Muito rápido para *busca exata* (=).
- **Ineficiente para intervalos** (BETWEEN, >, <) e ordenações.
- No MySQL, principalmente disponível na engine Memory MySQL 8.0 Docs.

```
1 CREATE TABLE Clientes (  
2     id      INT PRIMARY KEY AUTO_INCREMENT,  
3     email   VARCHAR(100)  
4 ) ENGINE=Memory;  
5  
6 CREATE INDEX idx_email_hash ON Clientes(email) USING HASH;  
7  
8 SELECT * FROM Clientes WHERE email = 'cliente45@dominio.com';
```

Pratique no MySQL

Use uma tabela populada com pelo menos 100 000 linhas (use `INSERT INTO ... SELECT` com `information_schema`).

- 1 Rode uma consulta com filtro em uma coluna sem índice. Anote o tempo (use `SET profiling=1;` ou cronômetro).
- 2 Crie um índice apropriado e rode a mesma consulta. Compare o tempo.
- 3 Use `EXPLAIN` antes e depois e mostre a diferença na coluna `type` e `key`.
- 4 Pense em uma situação em que adicionar índice seria *prejudicial* e explique por quê.

Atividade integrada – 60 minutos

- 1 Escolha uma das tabelas dos laboratórios anteriores (produtos, empregados etc.).
- 2 Popule até pelo menos 50 000 linhas usando `INSERT ... SELECT` com geração aleatória.
- 3 Cronometre 3 consultas: filtro simples, filtro + JOIN, agregação com `GROUP BY`.
- 4 Crie índices apropriados nas colunas-chave.
- 5 Repita as 3 consultas e compare os tempos.
- 6 Apresente um relatório com tabela “*antes/depois*” dos tempos e dos planos do `EXPLAIN`.

Recap

- 1 **Fundamentos:** BD, SGBD, modelos, MySQL e Workbench.
- 2 **Modelo relacional,** DDL e DML.
- 3 **DQL:** SELECT, WHERE, operadores lógicos, ORDER BY, LIMIT.
- 4 **Agregação:** COUNT, SUM, AVG, MIN, MAX, GROUP BY, HAVING.
- 5 **Subconsultas** básicas (em WHERE e FROM).
- 6 **Modelagem** conceitual e lógica + **mapeamento** para tabelas.
- 7 **JOINS** (INNER, LEFT, RIGHT, CROSS) com diagramas de Venn.
- 8 **Normalização** (1FN, 2FN, 3FN).
- 9 **Armazenamento e indexação** (introductório).

Próximos tópicos – BD II

- **Programação em banco:** *stored procedures, functions, cursors*.
- **Triggers:** automações dirigidas a eventos.
- **Views:** consultas armazenadas e simplificação de modelos.
- **Controle avançado de transações:** níveis de isolamento, bloqueios, deadlocks.
- **Administração de SGBD:** backup, replicação, tuning, segurança.
- **Tópicos avançados de modelagem:** FNBC, 4FN, OLAP vs OLTP, particionamento.

Mensagem final

A base que vocês construíram em BD I é o que torna BD II possível. **SQL declarativo bem feito é o que separa um programador júnior de um engenheiro de dados.**

Especificação

Em duplas, escolham um **tema** (loja virtual, biblioteca, clínica, escola, time de futebol...) e entreguem:

- 1 **DER conceitual** no brModelo.
- 2 **Modelo lógico** explicando o mapeamento (1:1 / 1:N / N:N).
- 3 **Script SQL** de criação (CREATE TABLEs + constraints + FKs), em 3FN.
- 4 **Carga de dados**: *seed* com pelo menos 30 linhas distribuídas entre as tabelas.
- 5 **10 consultas SQL** cobrindo: WHERE, LIKE, IN, BETWEEN, GROUP BY, HAVING, subconsulta, INNER JOIN, LEFT JOIN e uma consulta com 3 tabelas.
- 6 **Relatório PDF** com explicação textual de cada query.

Apresentação: 10 minutos (5 cada integrante) na semana 14.

Obrigado!

Bons estudos e até Banco de Dados II!

ronierison.maciел@pe.senac.br
github.com/0x03c1